

PRACTICA 07

Simulación de BD distribuidas multibase para MySql y Postgres en contexto localhost

Objetivo

Simular un ambiente distribuido con *SMBD* multibase con *MySql* y *Postgres* en servidores *localhost* con archivo de registros de actualizaciones para garantizar continuidad en las operaciones independientemente de si algún servidor está detenido o sin conexión utilizando programación Python.

Contexto

En un ambiente distribuido, se debe garantizar en todo momento que las operaciones siguen funcionando independientemente de si los servicios de algún servidor se encuentran detenidos o sin funcionar.

Esta PRACTICA 07 es una simulación de el comportamiento de un sistema distribuido mediante sistemas multibase, es continuidad principalmente de la PRCTICAS 05 con servidores *localhost*.

Primero se hacen consultas a ambos servidores, luego se hacen actualizaciones.

En Visual Studio Code:

1. Crear carpeta llamada *Práctica 05* y configurar entorno

1.1. Configurar entorno

```
python -m venv .venv  
  
.\.venv\Scripts\Activate.ps1
```

1.2. Instalar librerías

```
python -m pip install mysql-connector-python  
  
pip -m install mysql-connector-python psycopg2-binary
```

1.3. Crear archivo *requirements.txt*

```
pip freeze > requirements.txt
```

1.4. Identificar el archivo *CConexiones.py* y probar *CConexiones.py*

El archivo *CConexiones.py* contiene la clase que realiza las conexiones a *MySQL* y a *Postgres* respectivamente y el archivo *probar CConexiones.py* comprueba las conexiones y devuelve *msj* de conectado exitoso o Error en caso de que el servicio est[e detenido; ejemplo:

✓ Conexión MySQL exitosa

X Error de conexión PostgreSQL

O bien:

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 0
● 7> python.exe '.\probar CConexiones.py'
=====
PRUEBA DE CONEXIONES
=====
Usuario: rpizarro
Contraseña:
✓ Conexión MySQL exitosa
✓ Conexión PostgreSQL exitosa
○ (.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 0
7> █
```

2. Realizar consultas

Simulando que uno de los dos servicios se detiene por ejemplo *MySQL*, realizar consultas de tal manera que de manera transparente se responde a la petición de la consulta independientemente de cuál servidor responde.

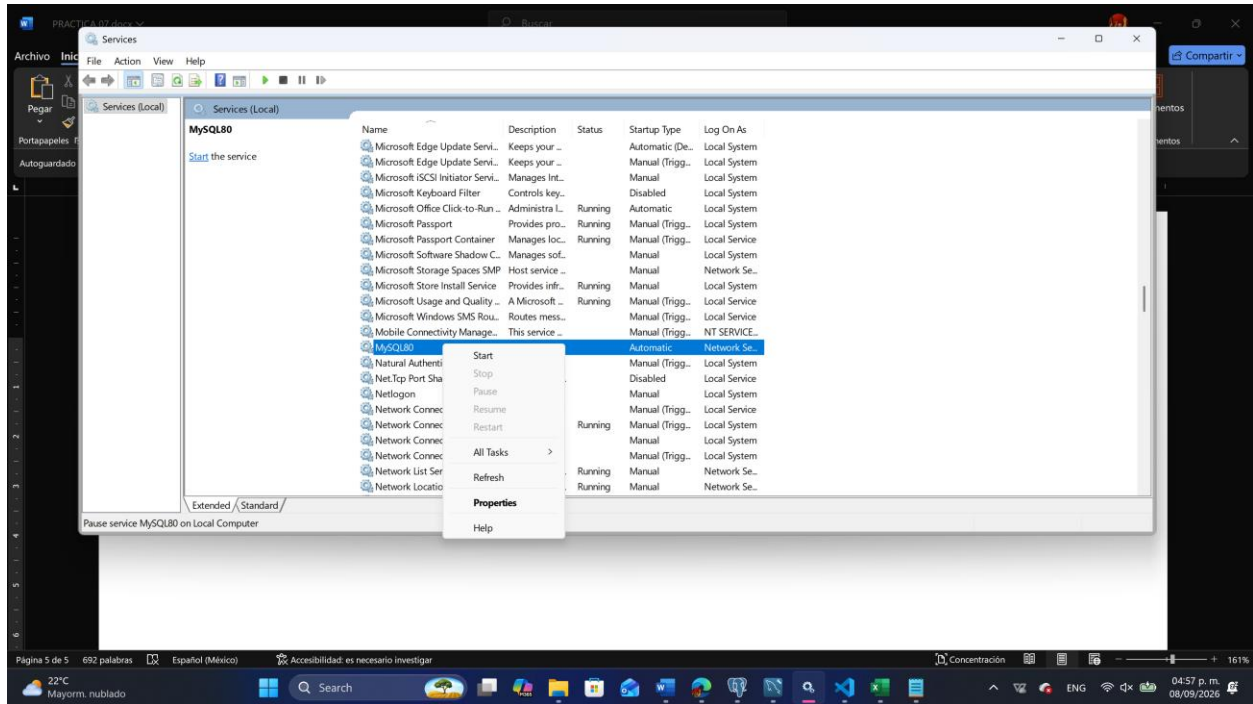
Puede haber estas combinaciones alternativas en relación a los servicios inicializados o detenidos cuando se ejecutan consultas al igual que en actualizaciones más adelante descritas:

- Que el servicio de *MySQL* esté detenido y *Postgres* en ejecución
- Que el servicio de *MySQL* esté en ejecución y *Postgres* detenido
- Que ambos servicios estén en ejecución o iniciados

- Que ambos servicios estén detenidos

2.1. Detener el servicio de *MySql* y *Postgres* en ejecución

En Windows, en el panel de control en servicios, detener *MySql* y solo queda funcionando el servicio de *Postgres*.



2.2. Lanzar una consulta

Consultar todas las ventas

```
select * from ventas
```

Con el usuario *rpizarro* que ya se tiene creado para trabajar tanto en *MySql* como en *Postgres* y con privilegios otorgados con anticipación se hace la consulta ejecutando el archivo de *Python*:

consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py

y que contiene este código:

```
from getpass import getpass
from CConexiones import Conexiones
# =====
# EJECUTAR CONSULTA
```

```

# =====
def ejecutar_consulta(conexion, sql):
    cursor = conexion.cursor()
    cursor.execute(sql)
    # Nombres de las columnas
    columnas = [columna[0] for columna in cursor.description]
    # Obtener registros
    registros = cursor.fetchall()
    cursor.close()
    return columnas, registros

# =====
# MOSTRAR RESULTADOS
# =====
def mostrar_resultados(columnas, registros):
    print("\n=====")
    print("          RESULTADO DE LA CONSULTA")
    print("=====\\n")
    # Encabezados
    for columna in columnas:
        print(f"{columna:<25}", end="")
    print()
    print("-" * (25 * len(columnas)))
    # Registros
    for registro in registros:
        for dato in registro:
            print(f"{str(dato):<25}", end="")
        print()
    print("\\nRegistros encontrados:", len(registros))

# =====
# CONSULTA DISTRIBUIDA
# =====
def consulta_distribuida(conexiones, sql):
    conexion_mysql = None
    conexion_postgres = None
    mysql_disponible = False
    postgres_disponible = False
    # =====
    # 1. VERIFICAR MYSQL
    # =====
    try:
        conexion_mysql = conexiones.conectar_MySQL("negocios")
        mysql_disponible = True
    except Exception:
        mysql_disponible = False
    # =====
    # 2. VERIFICAR POSTGRES
    # =====
    try:
        conexion_postgres = conexiones.conectar_Postgres("negocios")
        postgres_disponible = True
    except Exception:
        postgres_disponible = False
    # =====
    # 3. MOSTRAR ESTADO DE LOS SERVICIOS

```

```

# =====
print("\n=====")
print("          ESTADO DE LOS SERVICIOS")
print("=====")
if mysql_disponible:
    print("✓ Servicio MySQL disponible")
else:
    print("X Servicio MySQL detenido o no disponible")

if postgres_disponible:
    print("✓ Servicio PostgreSQL disponible")
else:
    print("X Servicio PostgreSQL detenido o no disponible")
# =====
# 4. REALIZAR CONSULTA
# =====
try:
    # -----
    # SI MYSQL ESTA DISPONIBLE
    # -----
    if mysql_disponible:
        columnas, registros = ejecutar_consulta(
            conexion_mysql,
            sql
        )
        print("\n✓ Consulta exitosa desde MySQL")
        mostrar_resultados(
            columnas,
            registros
        )
    # -----
    # SI MYSQL NO ESTA DISPONIBLE
    # USAR POSTGRESQL
    # -----
    elif postgres_disponible:
        columnas, registros = ejecutar_consulta(
            conexion_postgres,
            sql
        )
        print("\n✓ Consulta exitosa desde PostgreSQL")
        mostrar_resultados(
            columnas,
            registros
        )
    # -----
    # NINGUN SERVICIO ESTA DISPONIBLE
    # -----
    else:
        print("\n=====")
        print("          CONSULTA NO DISPONIBLE")
        print("=====")
        print("\nX MySQL no está disponible")
        print("X PostgreSQL no está disponible")

```

```

        print("\nNo fue posible realizar la consulta.")
except Exception as error:
    print("\n=====")
    print("          ERROR EN CONSULTA")
    print("=====")
    print("\nNo fue posible ejecutar la consulta SQL.")
    print("\nDetalle técnico:")
    print(error)
finally:
    # =====
    # CERRAR CONEXIONES
    # =====
    if conexion_mysql is not None:
        try:
            conexion_mysql.close()
        except:
            pass
    if conexion_postgres is not None:
        try:
            conexion_postgres.close()
        except:
            pass
# =====
# PROGRAMA PRINCIPAL
# =====
if __name__ == "__main__":
    print("=====")
    print("          CONSULTA DISTRIBUIDA")
    print("          MySQL - PostgreSQL")
    print("=====")
    # =====
    # USUARIO Y CONTRASEÑA
    # =====
    usuario = input("\nUsuario: ")
    password = getpass("Contraseña: ")
    conexiones = Conexiones(
        usuario=usuario,
        password=password
    )
    # =====
    # CAPTURAR CONSULTA
    # =====
    print("\nEscriba la consulta SQL")
    print("Solo se permiten instrucciones SELECT.")
    sql = input("\nSQL > ")
    # =====
    # VALIDAR SELECT
    # =====
    if sql.strip().upper().startswith("SELECT"):
        consulta_distribuida(
            conexiones,
            sql
        )
    else:
        print("\nX Solo se permiten instrucciones SELECT.")

```

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python.exe '.\consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py'
=====
CONSULTA DISTRIBUIDA
MySQL - PostgreSQL
=====

Usuario: rpizarro
Contraseña:

Escriba la consulta SQL
Solo se permiten instrucciones SELECT.

SQL > select * from ventas

=====
ESTADO DE LOS SERVICIOS
=====
X Servicio MySQL detenido o no disponible
✓ Servicio PostgreSQL disponible

✓ Consulta exitosa desde PostgreSQL

=====
RESULTADO DE LA CONSULTA
=====
id_venta      fecha                id_cliente            total
-----
1      2026-09-07 19:59:57.862260CLI00000000001      900.00
2      2026-09-07 19:59:57.862260CLI00000000002      3700.00
3      2026-09-07 19:59:57.862260CLI00000000003      1250.00
4      2026-09-07 19:59:57.862260CLI00000000004      1550.00
5      2026-09-07 19:59:57.862260CLI00000000005      3900.00

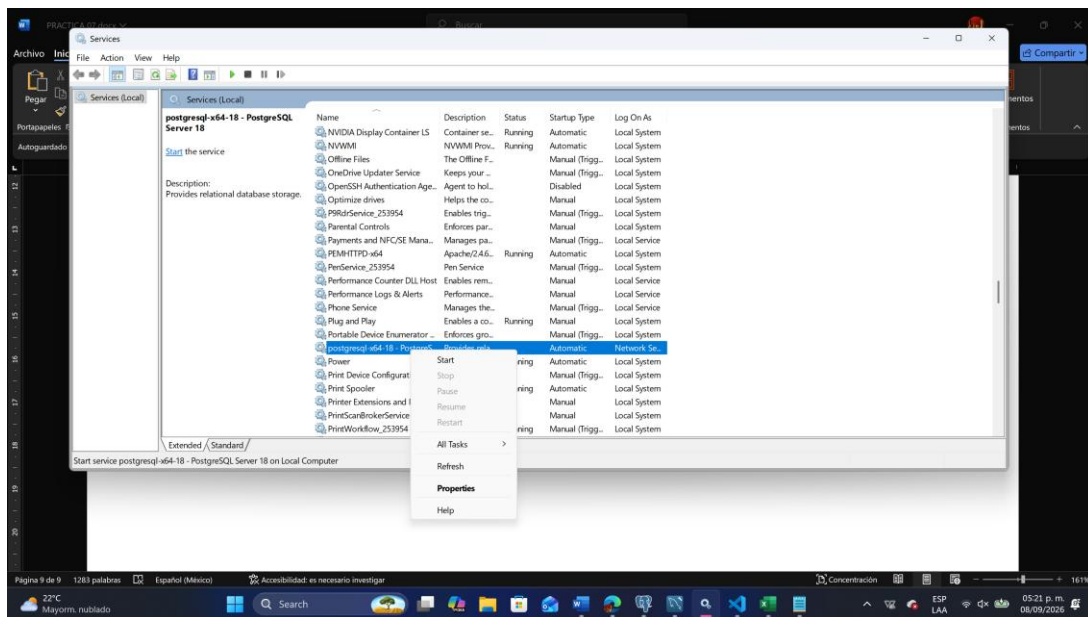
Registros encontrados: 5
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>
```

2.3. Detener el servicio de *Postgres* y *MySQL* en ejecución

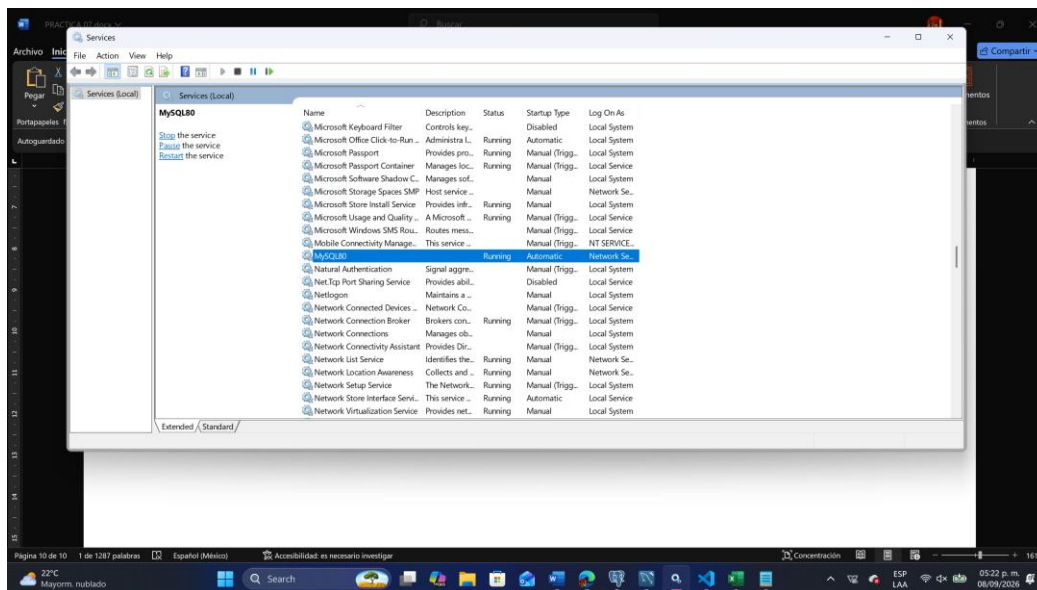
Ahora, invertir los estados de los servicios, nuevamente en el panel de control en servicios, detener el servicio *Postgres* y poner en ejecución el servicio de *MySQL* y lanzar la misma consulta.

```
select * from ventas
```

Detener *Postgres*:



Iniciar *MySql*:



Lanzar la consulta ejecutando nuevamente el archivo de Python

consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py

En modo terminal de *Visual Studio Code*:

```
python.exe '.\consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py'
```


Se observa que la consulta responde con *MySQL* en ejecución y *Postgres* detenido garantizando continuidad y transparencia de localización porque al responder la consulta no se sabe de donde proviene la respuesta, aunque en la simulación si se identifica de cual servicio se ejecuta.

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python.  
'.\consulta_distribuida_negocios_Mysql-Postgres_VER_2_PARA_PRACTICA_07.py'  
CONSULTA DISTRIBUIDA  
MySQL - PostgreSQL  
=====
```

Usuario: rpizarro
Contraseña:

Escriba la consulta SQL
Solo se permiten instrucciones SELECT.

SQL > select * from ventas

```
=====
```

ESTADO DE LOS SERVICIOS

```
=====
```

✓ Servicio MySQL disponible
X Servicio PostgreSQL detenido o no disponible

✓ Consulta exitosa desde MySQL

```
=====
```

RESULTADO DE LA CONSULTA

```
=====
```

id_venta	fecha	id_cliente	total
1	2026-09-07 18:13:07	CLI000000001	900.00
2	2026-09-07 18:13:07	CLI000000002	3700.00
3	2026-09-07 18:13:07	CLI000000003	1250.00
4	2026-09-07 18:13:07	CLI000000004	1550.00
5	2026-09-07 18:13:07	CLI000000005	3900.00

Registros encontrados: 5

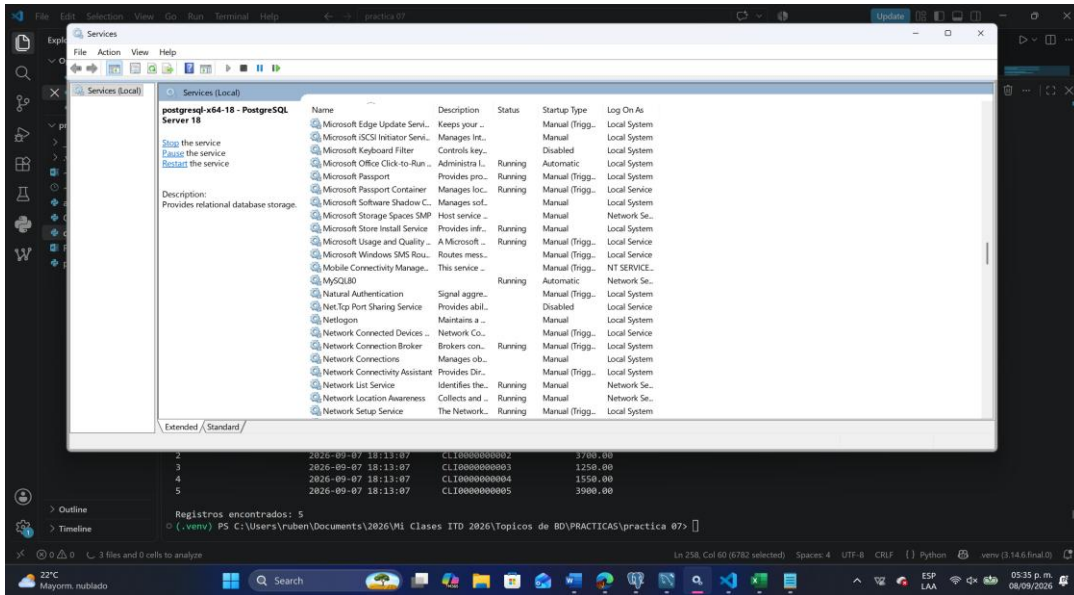
```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> █
```

2.4. Que ambos servicios estén en ejecución o iniciados

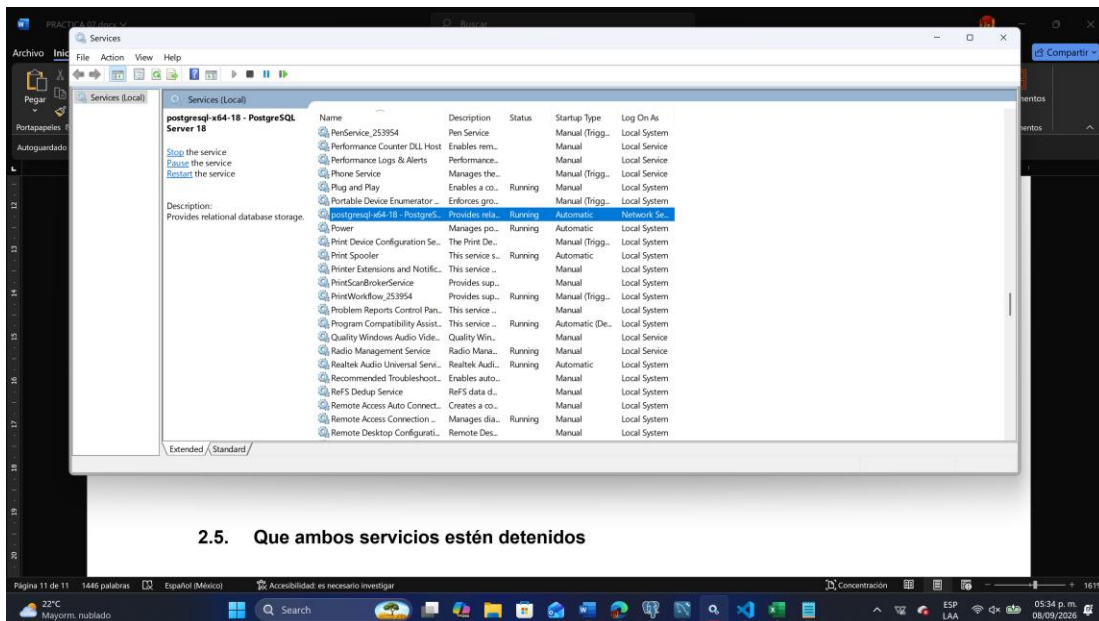
¿Qué sucede si ambos servicios están en ejecución?, la respuesta debiera ser que cualquiera puede responder, porque ambos SGBD tienen la misma información.

Garantizar que ambos servicios están en ejecución en el panel de control en la opción de servicios en Windows.

MySQL en ejecución:



Postgres en ejecución:

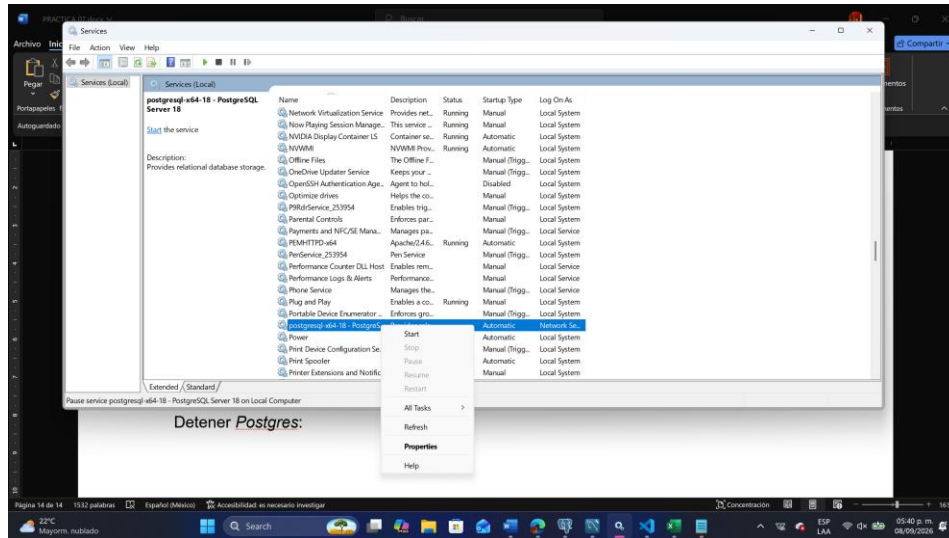


2.5. Que ambos servicios estén detenidos

Nuevamente ejecutar el archivo de *Python* y lanzar la consulta:

```
python.exe '.\consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py'
```

```
select * from ventas
```

Nuevamente lanzar la consulta con el usuario *rpizarro* ejecutando el programa

```
python.exe '.\consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py'
```

y la misma consulta o cualquier otra

```
select * from detalle_ventas
```

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python.exe '.\consulta distribuida negocios Mysql-Postgres VER 2 PARA PRACTICA 07.py'

=====
CONSULTA DISTRIBUIDA
MySQL - PostgreSQL
=====

Usuario: rpizarro
Contraseña:

Escriba la consulta SQL
Solo se permiten instrucciones SELECT.

SQL > select * from detalle_ventas

=====
ESTADO DE LOS SERVICIOS
=====
X Servicio MySQL detenido o no disponible
X Servicio PostgreSQL detenido o no disponible

=====
CONSULTA NO DISPONIBLE
=====

X MySQL no está disponible
X PostgreSQL no está disponible

No fue posible realizar la consulta.
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>
```

3. Realizar actualizaciones con servicios detenidos

Cuando hay actualizaciones de igual forma puede haber estas alternativas:

- Que ambos servicios estén en ejecución o iniciados
- Que el servicio de *MySQL* esté detenido y *Postgres* en ejecución
- Que el servicio de *MySQL* esté en ejecución y *Postgres* detenido
- Que ambos servicios estén detenidos

La pregunta es: ¿cómo se deben comportar el sistema de BD distribuido multibase cuando alguno de los dos servicios se encuentra detenido. Esto es la esencia de esta PRACTICA 07 con los servicios *localhost* de *MySQL* y *Postgres*.

La pregunta es, ¿qué sucede si alguno de los servidores de *MySQL* o *Postgres* en *localhost* se encuentra detenido o sin funcionar?, ¿qué estrategia seguir para que cuando el servicio del servidor que estaba detenido pueda continuar trabajando pero con la misma información que está en el otro servidor?

Al hacer alguna consulta, no importa si alguno de los servidores se encuentra detenido, la petición debe buscar el servidor alternativo para devolver la solicitud de la consulta.

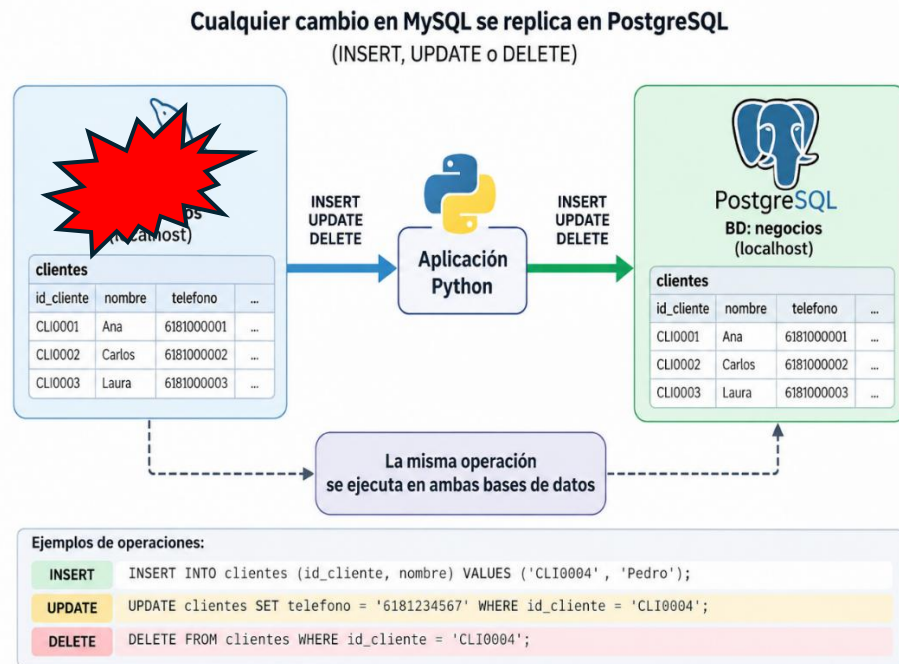
Sin embargo, en actualizaciones con operaciones *insert*, *update* o *delete*, cuando alguno de los servicios se encuentra detenido, se debe garantizar que las operaciones continúan y buscar un mecanismo que garantice que cuando se reinicie el servidor que está caído o detenido se actualice con las operaciones que se hicieron en el otro servicio mientras este estaba detenido.

Este mecanismo es utilizar un archivo texto al que se le identificará con el nombre *bitacora_caidas.txt* para guardar de manera temporal las operaciones cuando alguno de los servidores esté detenido, cuando se restablezca el servidor que estaba detenido, simplemente debe ejecutar las operaciones que le faltaron cuando no estaba activo y utilizará precisamente el archivo *bitacora_caidas.txt*.

Luego de actualizar la BD del servidor que estaba caído, el archivo texto se limpia y queda vacío porque su función fue temporal y registro las operaciones únicamente

cuando alguno de los dos servidores estaba caído o detenido. En caso de que ninguno esté caído, el archivo permanece vacío sin registros.

Por ejemplo, en la siguiente imagen:



- El servidor de MySQL está detenido o sin servicio
- Se hace una alta de un nuevo cliente *insert into clientes* por lo que el servicio de *MySQL* no responde pero el de *Postgres* si debe responder porque es el único que está activo.

```
INSERT INTO clientes (id_cliente,nombre,telefono,correo,direccion)
VALUES ('CLPruebaA','Prueba A','6181234567','pruebaA@email.com','Durango,
Dgo.')
```

- Para simular la estrategia y garantizar continuidad, se guarda en archivo texto almacenando la operación dado que un servicio está detenido.

bitacora_caidas_mysql.txt

```
INSERT INTO clientes (id_cliente,nombre,telefono,correo,direccion)
VALUES ('CLPruebaA','Prueba A','6181234567','pruebaA@email.com','Durango,
Dgo.')
```

- Luego se hace tal vez otra operación, tal vez se actualiza el precio a \$ 555 del producto identificado como PROD0000000000000005 de tabla productos:

```
update productos set precio = 555 where id_producto = 'PROD0000000000000005';
```

Ejecutar la operación en *Postgres* la estrategia debe almacenar la instrucción o instrucciones que no se pudieron ejecutar en *MySQL*, luego para que cuando se reinicie *MySQL* se actualicen las operaciones que le faltan haciendo uso del archivo *bitacora_caidas_mysql.txt* y con ello, garantizar que se tiene la misma información

- Entonces, se guarda en un archivo de texto *bitacora_caidas_mysql.txt* con lo siguiente

```
INSERT INTO clientes (id_cliente,nombre,telefono,correo,direccion) VALUES ('CLPruebaA','Prueba A','6181234567','pruebaA@email.com','Durango, Dgo.');
```

```
update productos set precio = 555 where id_producto = 'PROD0000000000000005';
```

Tal vez para bitácora, se sugiere almacenar fecha y hora y no se operación o más eficientemente en formato *JSON* semiestructurado en lugar de archivo texto estructurado.

La estrategia a seguir es la siguiente:

Estado <i>MySQL</i>	Estado <i>PostgreSQL</i>	INSERT / UPDATE / DELETE	Acción
Activo	Activo	Ejecutar	Se ejecuta en ambos. No se genera pendiente
Detenido	Activo	Ejecutar	Se ejecuta en <i>PostgreSQL</i> y se registra pendiente para <i>MySQL</i>
Activo	Detenido	Ejecutar	Se ejecuta en <i>MySQL</i> y se registra pendiente para <i>PostgreSQL</i>
Detenido	Detenido	No ejecutar	Se informa que no existe ningún servicio disponible

Para las operaciones de actualización, se utilizarán dos archivos temporales llamados:

```
bitacora_caidas_mysql.txt  
bitacora_caidas_postgres.txt
```

3.1. Ambos servicios en ejecución

Cuando ambos están en ejecución al hacer una actualización en un SMDB debe garantizar la actualización inmediata en el otro servidor por medio de transacciones desde Python (try except) y nativas en cada servicio (begin end y transacción commit).

En caso de alguna falla, *Python* coordina dos transacciones locales, una en *MySQL* y otra en *PostgreSQL*, utilizando try/except, COMMIT y ROLLBACK. La bitácora permite recuperar diferencias y alcanzar consistencia eventual ante fallos.

El programa para simular actualizaciones se llamará

```
actualizaciones Mysql-Postgres bitacoras PRACTICA07.py
```

El archivo está disponible en plataforma *classroom* y *github.com*

En la actualización ejecutar:

```
update cliente set teléfono = '6181111111' where id_cliente = 'CL10'
```

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python ".\actualizaciones Mysql-Postgres bitacoras PRACTICA07.py"  
SQL > update clientes set telefono = '6181111111' where id_cliente = 'CL10'  
  
=====  
ESTADO DE LOS SERVICIOS  
=====  
✓ MySQL disponible  
✓ PostgreSQL disponible  
  
Ambos servicios están disponibles.  
Ejecutando transacción coordinada...  
✓ MySQL aceptó la operación  
✓ PostgreSQL aceptó la operación  
✓ COMMIT realizado en MySQL  
✓ COMMIT realizado en PostgreSQL  
  
=====  
TRANSACCIÓN COORDINADA EXITOSA  
=====  
✓ MySQL actualizado  
✓ PostgreSQL actualizado  
○ No se generó operación pendiente  
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> █
```

Al consultar en ambos servicios, debe estar la actualización del teléfono en el cliente 'CL10':

MySQL:

The screenshot shows the MySQL Query Editor interface. The query window contains the following SQL commands:

```
1 use negocios;  
2  
3 -- update productos set precio = 399 where id_producto = 'PROD0000000000000001';  
4  
5 select * from clientes;
```

The result grid displays the following data:

id_cliente	nombre	telefono	correo	direccion
CL10	RUBEN PIZARRO	6181111111	rpizarro@itdurango.edu.mx	SUR
CLI0000000001	Ana Martinez	6181000001	ana@email.com	Durango, Dgo.
CLI0000000002	Carlos Ramirez	6181000002	carlos@email.com	Durango, Dgo.
CLI0000000003	Laura Hernandez	6181000003	laura@email.com	Gomez Palacio, Dgo.
CLI0000000004	Jose Rodriguez	6181000004	jose@email.com	Lerdo, Dgo.
CLI0000000005	Maria Lopez	6181000005	maria@email.com	Durango, Dgo.
CLI0000000006	Pedro Sanchez	6181234567	pedro@email.com	Durango, Dgo.
NULO	NULO	NULO	NULO	NULO

Y Postgres:

The screenshot shows the PostgreSQL Query Editor interface. The query window contains the following SQL command:

```
1 select * from clientes;  
2  
3
```

The data output table displays the following data:

id_cliente [PK] character (13)	nombre character varying (100)	telefono character varying (15)	correo character varying (100)	direccion character varying (150)	
1	CLI0000000001	Ana Martinez	6181000001	ana@email.com	Durango, Dgo.
2	CLI0000000002	Carlos Ramirez	6181000002	carlos@email.com	Durango, Dgo.
3	CLI0000000003	Laura Hernandez	6181000003	laura@email.com	Gomez Palacio, Dgo.
4	CLI0000000004	Jose Rodriguez	6181000004	jose@email.com	Lerdo, Dgo.
5	CLI0000000005	Maria Lopez	6181000005	maria@email.com	Durango, Dgo.
6	CLI0000000006	Pedro Sanchez	6181234567	pedro@email.com	Durango, Dgo.
7	CL10	RUBEN PIZARRO	6181111111	rpizarro@itdurango.edu...	SUR

3.2. Que el servicio de *MySql* esté detenido y *Postgres* en ejecución

3.2.1. Ejecutar Actualización

```
python ".\actualizaciones Mysql-Postgres bitacoras PRACTICA07.py"
```

Modificar el precio del producto a 333.33

```
id_producto = 'PROD0000000000000001'
```

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python ".\actualizaciones Mysql-Postgres bitacoras PRACTICA07.py"

Escriba la instrucción SQL.
Operaciones permitidas: INSERT, UPDATE o DELETE

SQL > update productos set precio = 333.33 where id_producto = 'PROD0000000000000001'

=====
ESTADO DE LOS SERVICIOS
=====
X MySQL detenido o no disponible
✓ PostgreSQL disponible

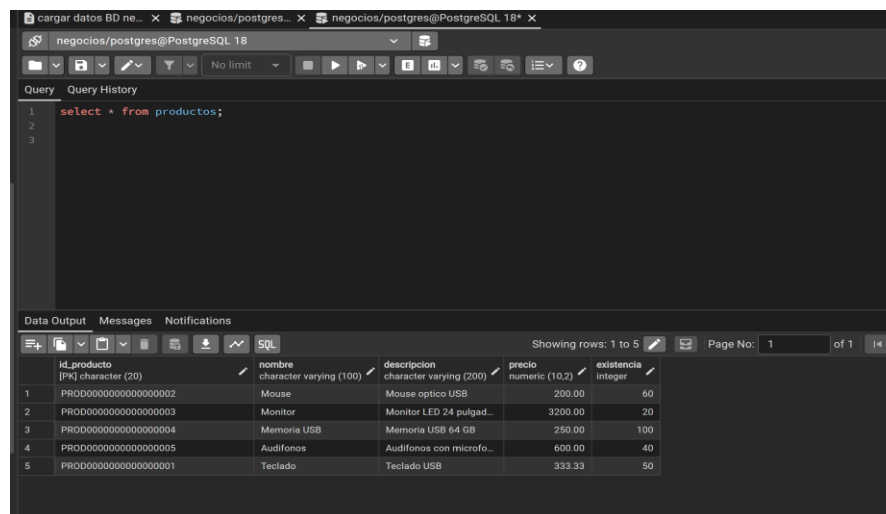
MySQL está detenido.
La operación continuará en PostgreSQL.
✓ Operación realizada en PostgreSQL
✓ COMMIT realizado en PostgreSQL

✓ PostgreSQL quedó actualizado
⚠ MySQL quedó pendiente
✓ Pendiente guardado en bitacora_caidas_mysql.txt
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>
```

Hay que verificar dos cosas:

- Que exista la actualización en productos en *Postgres*

En consultas en Postgres



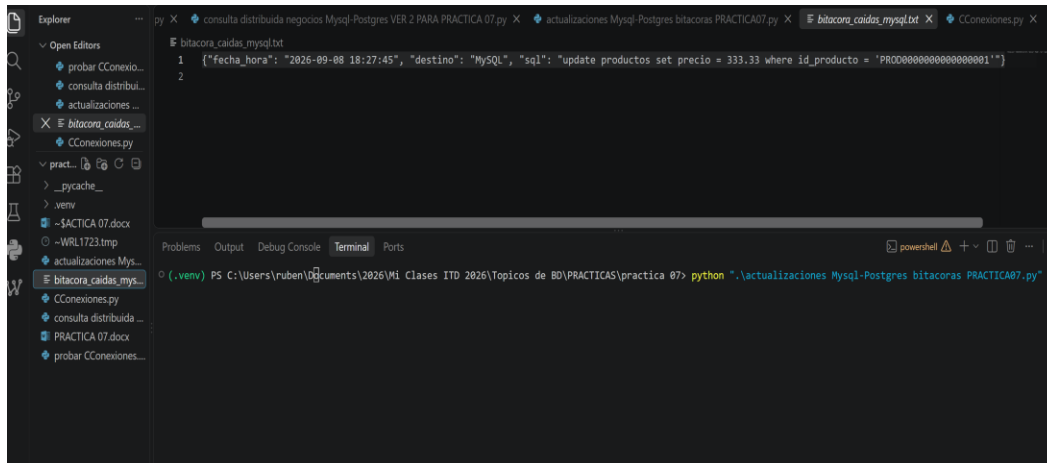
The screenshot shows a PostgreSQL query editor with the following query executed:

```
select * from productos;
```

The results are displayed in a table with 5 rows and 5 columns:

	id_producto	nombre	descripcion	precio	existencia
1	PROD0000000000000002	Mouse	Mouse optico USB	200.00	60
2	PROD0000000000000003	Monitor	Monitor LED 24 pulgad...	3200.00	20
3	PROD0000000000000004	Memoria USB	Memoria USB 64 GB	250.00	100
4	PROD0000000000000005	Audifonos	Audifonos con microfo...	600.00	40
5	PROD0000000000000001	Teclado	Teclado USB	333.33	50

- Que exista la bitácora con el registro almacenado para que se actualice posteriormente *MySQL*. Para ello hay que verificar el archivo *bitacora_caidas_mysql.txt*



Es un archivo en formato JSON con este contenido:

```
{"fecha_hora": "2026-09-08 18:27:45", "destino": "MySQL", "sql": "update productos set precio = 333.33 where id_producto = 'PROD0000000000000001'"}}
```

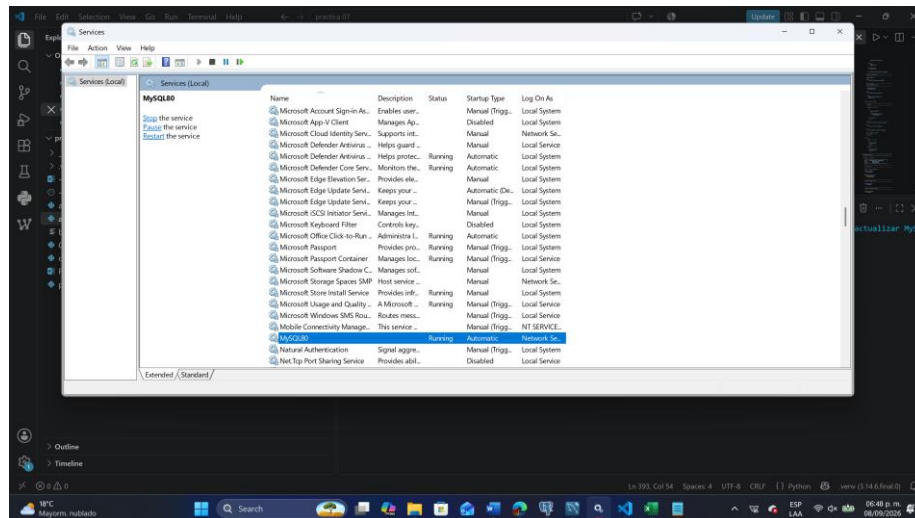
Que deberá usarse para actualizar *MySQL*

3.2.2. Actualizar *MySQL* con el archivo de bitácora

Se debe actualizar *MySQL* toda vez iniciado el servicio haciendo uso del archivo:

bitacora_caidas_Mysql.txt

Reiniciar el servicio de *MySQL* simulando que este se ha reestablecido y continua en operando:



Se debe ejecutar el programa *Python*

actualizar MySQL-Postgres desde bitacoras.py

```

Problems Output Debug Console Terminal Ports
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python.exe '.\actualizar MySQL-Postgres desde bitacoras.py'
=====
ACTUALIZANDO MYSQL
=====
Operaciones pendientes: 1

Operación: 1
Fecha/Hora: 2026-09-08 18:27:45
SQL: update productos set precio = 333.33 where id_producto = 'PROD0000000000000001'
Operación ejecutada correctamente.

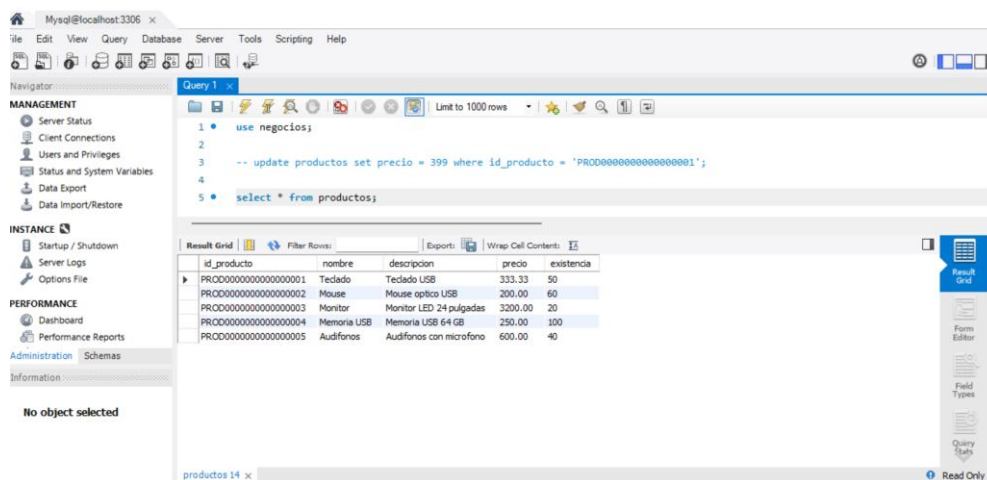
COMMIT realizado en MySQL.
MySQL quedó actualizado.
bitacora_caidas_Mysql.txt quedó vacía.

PostgreSQL:
No existen operaciones pendientes.

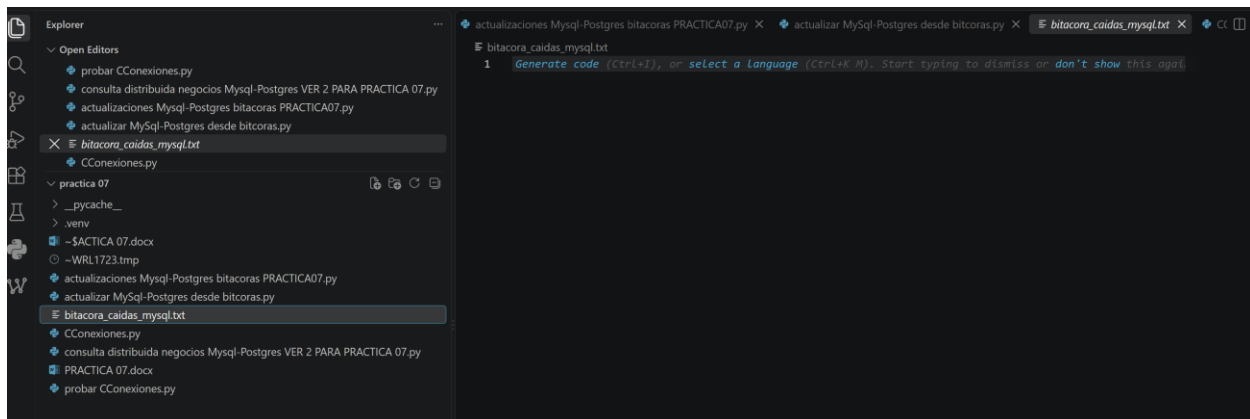
=====
PROCESO FINALIZADO
=====
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>

```

Ahora consultar en *MySQL*/ debe existir actualizado el precio del producto

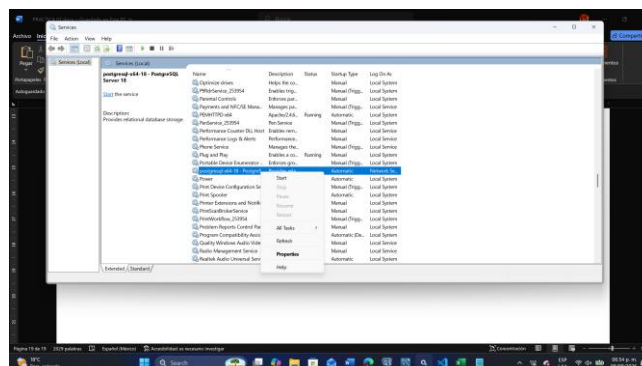


El contenido de los archivos bitácoras, en este caso `bitacora_caidas_mysql.txt` debe estar vacío y limpio y ambos BD tanto de *MySql* como de *Postgres* ahora tienen la misma información.



3.3. Que el servicio de *MySql* esté en ejecución y *Postgres* detenido

Ahora invertir los servicios, dejar en ejecución solo *MySql* y detener *Postgres*:



3.3.1. Ejecutar Actualización

```
python ".\actualizaciones Mysql-Postgres bitacoras PRACTICA07.py"
```

Agregar un nuevo producto mesa de oficina,

- nombre: 'Mesa de oficina ejecutiva'
- descripcion 'Mesa de oficina ejecutiva, mesa y silla'
- precio: 2500
- existencia: 30

```
insert into productos (id_producto, nombre, descripcion, precio, existencia) values ('MESAOF',  
'Mesa de oficina ejecutiva', 'Mesa de oficina ejecutiva, mesa y silla', 2500, 30)
```

Ejecutar nuevamente el programa de *Python*:

```
python ".\actualizaciones Mysql-Postgres bitacoras PRACTICA07.py"
```

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python ".\actualizaciones M  
sql-Postgres bitacoras PRACTICA07.py"  
ACTUALIZACIONES DISTRIBUIDAS  
MySQL - PostgreSQL  
PRACTICA 07  
=====
```

Usuario: rpizarro
Contraseña:

Escriba la instrucción SQL.
Operaciones permitidas: INSERT, UPDATE o DELETE

SQL > insert into productos (id_producto, nombre, descripcion, precio, existencia) values ('MESAOF', 'Mesa de oficina ejecuti
va', 'Mesa de oficina ejecutiva, mesa y silla', 2500, 30)

```
=====
```

ESTADO DE LOS SERVICIOS

```
=====
```

✓ MySQL disponible
X PostgreSQL detenido o no disponible

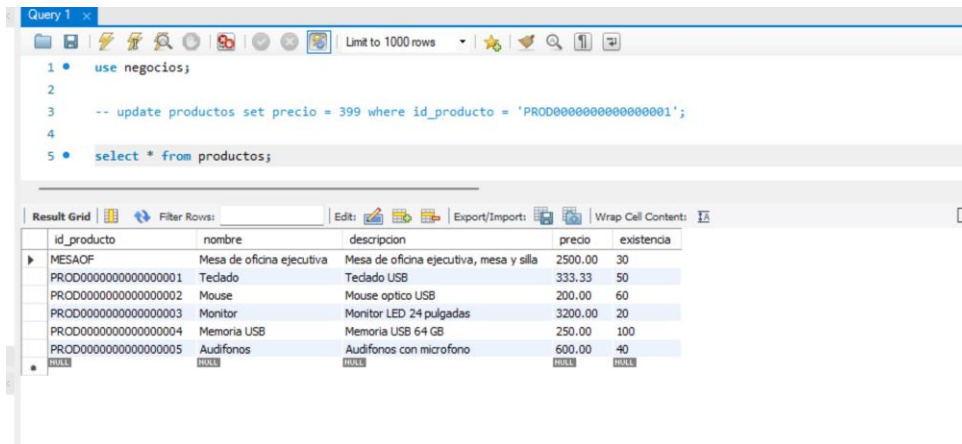
PostgreSQL está detenido.
La operación continuará en MySQL.
✓ Operación realizada en MySQL
✓ COMMIT realizado en MySQL

✓ MySQL quedó actualizado
Δ PostgreSQL quedó pendiente
✓ Pendiente guardado en bitacora_caidas_postgres.txt

```
○ (.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>
```

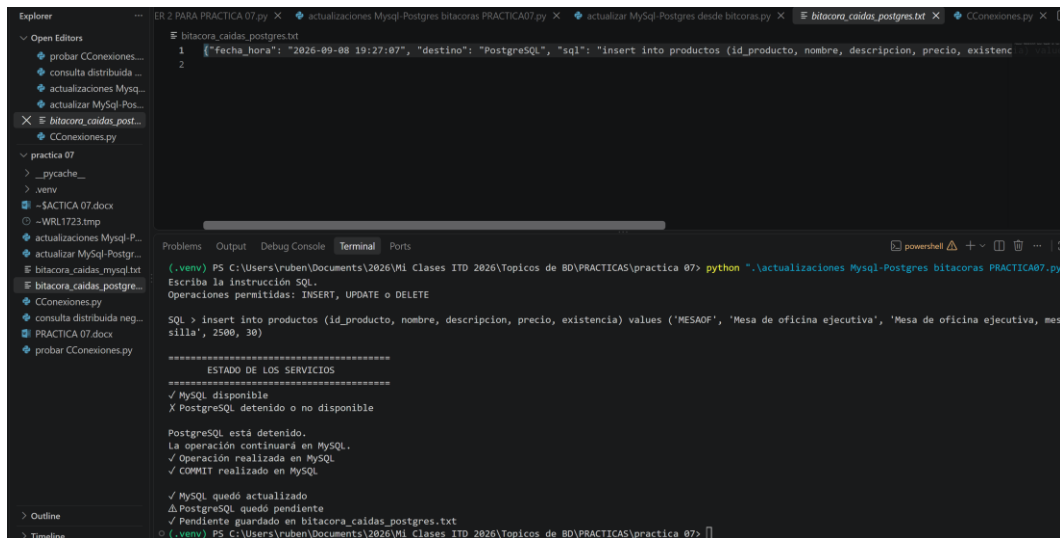
Verificar dos cosas:

- Que se actualice MySQL



- Que el archivo de bitácora se agregue el registro para cuando se reinicie *Postgres*

bitacora_caidas_postgres.txt



Es un archivo en formato JSON con este contenido:

```

{"fecha_hora": "2026-09-08 19:27:07", "destino": "PostgreSQL", "sql": "insert into productos (id_producto, nombre, descripcion, precio, existencia) values ('MESAOF', 'Mesa de oficina ejecutiva', 'Mesa de oficina ejecutiva, mesa y silla', 2500, 30)"}

```

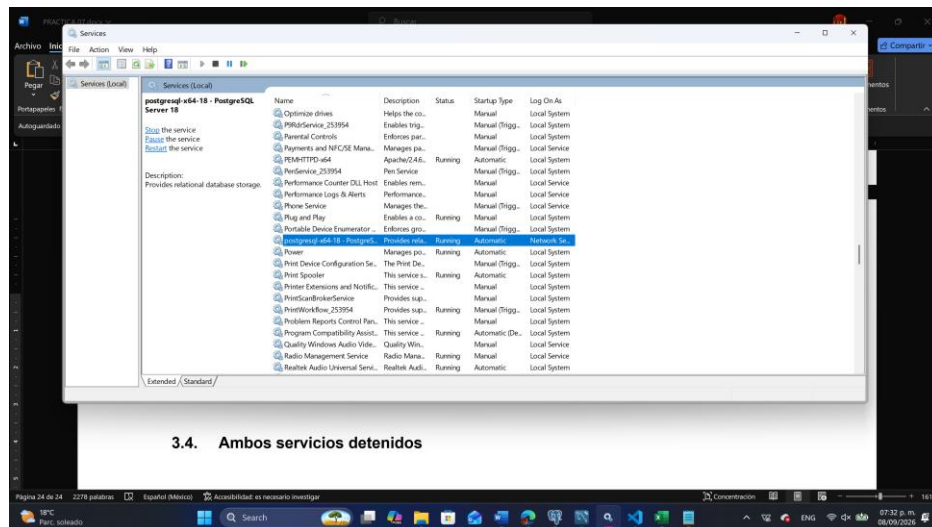
Que deberá usarse para actualizar *Postgres*

3.3.2. Actualizar *Postgres* con el archivo de bitácora

Se debe actualizar *Postgres* toda vez iniciado el servicio haciendo uso del archivo:

bitacora_caidas_postgres.txt

Reiniciar el servicio de *Postgres* simulando que este se ha reestablecido y continua en operando:



Se debe ejecutar el programa *Python*

actualizar MySQL-Postgres desde bitacoras.py

en la terminal de Visual Studio Code:

Python '.\actualizar MySQL-Postgres desde bitcoras.py'

```
Problems Output Debug Console Terminal Ports
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Temas de BD\PRACTICAS\practica 07> Python '.\actualizar MySQL-Postgres desde bitcoras.py'
DESD E BITACORAS
PRACTICA 07
=====

Usuario: rpizarro
Contrasea:

MySQL:
No existen operaciones pendientes.

=====
ACTUALIZANDO POSTGRESQL
=====
Operaciones pendientes: 1

Operación: 1
Fecha/Hora: 2026-09-08 19:27:07
SQL: insert into productos (id_producto, nombre, descripcion, precio, existencia) values ('MESAOF', 'Mesa de oficina ejecutiva', 'Mesa de oficina ejecutiva, mesa y
illa', 2500, 30)
Operación ejecutada correctamente.

COMMIT realizado en PostgreSQL.
PostgreSQL quedó actualizado.
bitacora_caidas_Postgres.txt quedó vacia.

=====
PROCESO FINALIZADO
=====
```

Ahora consultar en *Postgres* debe existir agregado el producto de la mesa ejecutiva.

The screenshot shows a PostgreSQL client window titled 'negocios/postgres@PostgreSQL 18'. The query editor contains the command: `select * from productos;`. The results pane displays a table with 6 rows and 5 columns: `id_producto` (PK), `nombre`, `descripcion`, `precio`, and `existencia`. The data is as follows:

id_producto	nombre	descripcion	precio	existencia
PROD00000000000000000002	Mouse	Mouse optico USB	200.00	60
PROD00000000000000000003	Monitor	Monitor LED 24 pulgadas	3200.00	20
PROD00000000000000000004	Memoria USB	Memoria USB 64 GB	250.00	100
PROD00000000000000000005	Audifonos	Audifonos con microfono	600.00	40
PROD00000000000000000001	Teclado	Teclado USB	333.33	50
MESAOF	Mesa de oficina ejecuti...	Mesa de oficina ejecutiva, mesa y s...	2500.00	30

3.4. Ambos servicios detenidos

Cuando ambos servicios están detenido, caídos o apagados, no debe efectuarse nada solo recibir mensaje de que no hay servicios:

```
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07> python ".\actualizaciones MySQL-Postgres bitacoras PRACTICA07.py"
PRACTICA 07
=====
Usuario: rpizarro
Contraseña:

Escriba la instrucción SQL.
Operaciones permitidas: INSERT, UPDATE o DELETE

SQL > update productos set precio = 1550 where id_producto = 'MESAOF'

=====
ESTADO DE LOS SERVICIOS
=====
X MySQL detenido o no disponible
X PostgreSQL detenido o no disponible

=====
OPERACIÓN NO DISPONIBLE
=====
X MySQL está detenido
X PostgreSQL está detenido.
La operación NO fue realizada.
No se agregó ninguna operación a las bitácoras.
(.venv) PS C:\Users\ruben\Documents\2026\Mi Clases ITD 2026\Topicos de BD\PRACTICAS\practica 07>
```

FIN DE LA PRACTICA 07